

Annexe : extension empirique — univers élargi et protocole train/validation/test

Summer Project : Du modèle à la stratégie — un cadre jump-diffusion pour le pairs trading

Cette annexe documente une extension empirique menée après la rédaction du rapport principal (`report.tex`), dans le prolongement direct des limites identifiées en 4.7–4.8 de celui-ci : l'échelle de l'étude originale (3 paires, une seule fenêtre de trading) ne permettait pas d'établir la significativité statistique du filtrage par sauts, et le Deflated Sharpe Ratio ($DSR = 0,155$) suggérait que le gain observé (+0,073 contre +0,036) était possiblement porté par un petit nombre de trades pivots plutôt que par un effet structurel robuste.

L'objectif ici est de vérifier si un protocole plus large et plus rigoureux — univers de paires élargi, fenêtre de backtest étendue, calibration hors-échantillon — confirme, nuance ou infirme cette lecture, **sans retomber dans le biais de sélection multiple que 4.7 avait lui-même détecté.**

A.1 — Motivation et démarche

Trois leviers ont été explorés dans l'ordre suivant :

- un **univers de paires élargi** : 88 paires candidates intra-secteur (12 secteurs, 51 titres du S&P 500 : paiements, énergie, retail, banques, biens de consommation, pharma, télécoms, compagnies aériennes, semi-conducteurs, assurance, promoteurs immobiliers, utilities), screenées par test ADF sur la fenêtre de formation puis retenues après correction de Benjamini–Hochberg (FDR, $q = 0,10$) pour contrôler le taux de fausses découvertes inhérent à un criblage sur autant de candidats — exactement la procédure recommandée mais non appliquée en 4.5 du rapport ;
- une **fenêtre de backtest étendue** : formation 2009–2013, trading 2014–2024 (11 ans, contre 6 ans initialement), pour réduire la sensibilité du résultat à un seul régime de marché ;
- une **sortie anticipée sur détection de saut** : la stratégie filtrée utilise π_t non seulement pour bloquer l'entrée (comme en 4.3 du rapport) mais aussi pour clore une position en cours dès que π_t dépasse un second seuil π_{exit} , même si le z -score n'est pas revenu à zéro — l'extension que le dépôt mentionnait déjà comme testée dans `backtest_4_6.py` sans être réconciliée avec les chiffres du rapport.

Point méthodologique central : les seuils ($\kappa, \pi^*, \pi_{\text{exit}}$) ne sont *jamais* choisis en lisant leur performance sur la période d'évaluation finale. Un bloc de **validation** (2014–2018) sert à la calibration par grid search ; un bloc de **test**, disjoint et jamais consulté pendant la calibration (2019–2024, soit la fenêtre de trading originale du rapport), sert à l'évaluation, en un seul passage. C'est le protocole en trois blocs déjà esquissé en 4.4 du rapport mais jamais mis en œuvre dans le corps de celui-ci.

A.2 — Criblage de l'univers élargi

Sur les 88 paires candidates, 31 passent le seuil brut $p < 0,05$ et **27 survivent à la correction FDR** ($q = 0,10$) — l'univers retenu pour la suite. Fait notable et cohérent avec la section 4.6 du rapport : *aucune* des trois paires originales (MA/V, XOM/CVX, HD/LOW) ne passe le seuil brut sur cette fenêtre de formation étendue, ce qui confirme, sur un échantillon indépendant, la fragilité déjà signalée de leur statut de paires cointégrées.

Paires retenues (27) : CVX/COP, HD/WMT, TGT/WMT, TGT/COST, JPM/WFC, JPM/C, BAC/C, KO/PG, PEP/PG, PG/CL, PG/KMB, CL/KMB, PFE/LLY, VZ/T, MET/PRU, MET/AIG, PRU/AIG,

PRU/TRV, AIG/ALL, AIG/TRV, ALL/TRV, DHI/LEN, DHI/PHM, DHI/NVR, LEN/NVR, DUK/AEP, AEP/D.

A.3 — Calibration et résultat hors-échantillon

La grille $\kappa \in \{1,5; 2,0; 2,5\}$, $\pi^* \in \{0,3; 0,4; 0,5\}$, $\pi_{\text{exit}} \in \{0,5; 0,6; 0,7; 0,8\}$ ($\pi_{\text{exit}} \geq \pi^*$) est évaluée sur la validation (2014–2018) ; la configuration retenue par argmax du Sharpe filtré est $\kappa = 2,5$, $\pi^* = 0,4$, $\pi_{\text{exit}} = 0,8$. Appliquée telle quelle, sans deuxième passage, au test (2019–2024) :

Stratégie	Sharpe annualisé	Nb. trades	Durée moy. (j)
Naïve (2.7)	0,323	413	39,5
Filtrée (4.3 + sortie sur saut)	0,343	442	28,2

Sur les 442 trades filtrés, 140 se terminent par une sortie anticipée sur détection de saut plutôt que par un retour du z -score à zéro. Ce mécanisme explique un fait qui peut surprendre à première vue : la stratégie filtrée compte *plus* de trades que la naïve (442 contre 413), alors que le filtre bloque toujours une partie des entrées. La sortie anticipée libère la position plus tôt (durée moyenne 28,2 jours contre 39,5), ce qui permet davantage de ré-entrées sur la même paire pendant la fenêtre de test ; l’effet net favorise donc le nombre de trades malgré le filtrage à l’entrée.

Le test de Jobson–Korkie/Memmel entre les deux séries de rendements donne $z = 0,103$ ($p = 0,918$) : l’écart n’est pas statistiquement significatif.

A.4 — Tableau récapitulatif de toutes les variantes testées

Configuration	Sharpe naïve	Sharpe filtrée	Trades naïve	Trades filtrée
Référence (rapport, 3 paires, 2019–2024)	0,036	0,073	73	69
Fenêtre étendue seule (3 paires, 2014–2024)	0,120	0,100	133	127
Univers élargi + fenêtre étendue (27 paires, équipondéré)	0,112	0,082	1144	1106
Univers élargi, pondération inverse-vol	0,186	0,143	1144	1106
Ré-estimation glissante annuelle (3 paires)	−0,002	−0,023	120	115
Calibration train/validation/test (27 paires)	0,323	0,300	413	372
+ sortie anticipée sur saut détecté	0,323	0,343	413	442

A.5 — Lecture honnête du résultat

Deux constats, tenus ensemble plutôt que l’un sans l’autre. D’une part, le Sharpe de la stratégie filtrée passe de 0,073 (rapport original, 3 paires, biais de sélection non contrôlé) à 0,343 (27 paires, protocole train/validation/test, aucune lecture de la réponse sur le test) — une hausse obtenue sans p-hacking, puisque la configuration a été figée avant d’observer la période d’évaluation. Le nombre de signaux passe de 69 à 442. D’autre part, l’écart entre stratégie filtrée et stratégie naïve, qui était le résultat central de la section 4.6 du rapport (+0,073 contre +0,036, un quasi-doublement), se réduit ici à un avantage ténu et non significatif (0,343 contre 0,323). Des variantes testées mais non retenues comme résultat final (univers élargi sans sortie sur saut, ré-estimation glissante annuelle) donnent tantôt un léger désavantage pour la filtrée, tantôt un résultat proche de zéro pour les deux stratégies — le signe de l’effet est instable, jamais son ordre de grandeur n’est net.

Cette instabilité corrobore, à plus grande échelle, ce que le Deflated Sharpe Ratio de la section 4.7 avait déjà détecté sur l’échantillon original ($DSR = 0,155$) : l’avantage du filtrage par sauts, aussi intuitif soit son fondement théorique (4.1–4.3), ne se manifeste pas comme un effet large et robuste une fois l’échantillon élargi et le protocole de calibration correctement cloisonné. La hausse du Sharpe absolu est réelle et défendable ; l’amélioration relative du filtrage sur la naïve, elle, reste dans la marge du bruit d’échantillonnage —

exactement la conclusion nuancée déjà formulée en 4.8 du rapport, renforcée ici par un test indépendant plutôt que contredite.

A.6 — Diversification : pourquoi trader plusieurs paires à la fois aide

La corrélation moyenne entre les rendements quotidiens des 27 paires n'est que de 0,046 — quasiment nulle, parce que chaque paire est un pari idiosyncratique sur l'écart entre deux titres précis. Avec une corrélation aussi faible, la formule classique de diversification de portefeuille $SR_{\text{portefeuille}} \approx SR_{\text{moyen}} \times \sqrt{N/(1 + (N-1)\bar{\rho})}$ prédit un Sharpe théorique de 0,186 pour un Sharpe individuel moyen de seulement 0,053 par paire — cohérent avec le Sharpe réalisé en pondération inverse-vol (0,186, cf. tableau A.4). Attention cependant : 7 des 27 paires retenues (MET/PRU, MET/AIG, PRU/AIG, PRU/TRV, AIG/ALL, AIG/TRV, ALL/TRV) sont toutes construites à partir des 4 mêmes titres du secteur assurance, donc elles partagent beaucoup de variance entre elles malgré des noms différents — la vraie diversification vient surtout de la variété des *secteurs*, pas juste du nombre brut de paires.

A.7 — Code

Le code complet est également disponible dans `code/` (`yahoo_dl.py`, `expanded_universe_backtest.py`, `validated_config_train_test.py`, `exit_on_jump_filter.py`, `rolling_reestimation.py`). Il est reproduit intégralement ci-dessous pour que cette annexe soit autonome.

A.7.1 — `yahoo_dl.py` (téléchargeur de prix sans dépendance à `yfinance`)

```

1  import time, os, json
2  import requests
3  import pandas as pd
4
5  CACHE_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "data_cache")
6  HEADERS = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
7           Gecko) Chrome/120.0 Safari/537.36"}
8
9  def _to_unix(date_str):
10     return int(pd.Timestamp(date_str, tz="UTC").timestamp())
11
12 def download_close(ticker, start, end, retries=4):
13     os.makedirs(CACHE_DIR, exist_ok=True)
14     cache_path = os.path.join(CACHE_DIR, f"{ticker}_{start}_{end}.csv")
15     if os.path.exists(cache_path):
16         s = pd.read_csv(cache_path, index_col=0, parse_dates=True)["close"]
17         return s
18     url = f"https://query1.finance.yahoo.com/v8/finance/chart/{ticker}"
19     params = dict(period1=_to_unix(start), period2=_to_unix(end), interval="1d", events="div,splits")
20     last_err = None
21     for attempt in range(retries):
22         try:
23             r = requests.get(url, headers=HEADERS, params=params, timeout=20)
24             if r.status_code == 200:
25                 d = r.json()
26                 res = d["chart"]["result"][0]
27                 ts = res["timestamp"]
28                 adj = res["indicators"]["adjclose"][0]["adjclose"]
29                 idx = pd.to_datetime(ts, unit="s", utc=True).tz_convert("America/New_York").normalize()
30                 .tz_localize(None)
31                 s = pd.Series(adj, index=idx, name="close").dropna()
32                 s = s[~s.index.duplicated(keep="first")]
33                 s.to_csv(cache_path, header=True)
34                 time.sleep(0.4)
35                 return s
36             else:
37                 last_err = f"HTTP {r.status_code}"
38         except Exception as e:
39             last_err = str(e)
40             time.sleep(1.5 * (attempt + 1))
41     raise RuntimeError(f"Failed to download {ticker}: {last_err}")

```

A.7.2 — expanded_universe_backtest.py (criblage FDR + backtest agrégé)

```

1  """
2  Extension du cadre pairs-trading + jump-diffusion (ArthurK67/pairs-trading-jump-diffusion) :
3      - univers de paires elargi (9 paires intra-secteur au lieu de 3)
4      - fenetre de backtest etendue (formation 2009-2013, trading 2014-2024 au lieu de 2014-2018/2019-2024)
5      - re-estimation glissante annuelle (option ROLLING)
6  Reutilise la logique de estimate_formation()/simulate()/deflated_sharpe_ratio() de robustness_4_7.py,
7  seule la source de donnees change (Yahoo chart API directe, yfinance etant bloqué sur ce reseau).
8  """
9  import sys, math, os
10 import numpy as np
11 import pandas as pd
12 from scipy.optimize import minimize
13 from scipy.stats import norm, chi2
14 from scipy.special import factorial
15 from statsmodels.tsa.stattools import adfuller
16 import statsmodels.api as sm
17
18 sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
19 from yahoo_dl import download_close
20
21 np.random.seed(0)
22
23 SECTOR_TICKERS = {
24     "Paielements": ["MA", "V", "AXP"],
25     "Energie": ["XOM", "CVX", "COP", "OXY"],
26     "Retail": ["HD", "LOW", "TGT", "WMT", "COST"],
27     "Banques": ["JPM", "BAC", "WFC", "C", "USB"],
28     "Staples": ["KO", "PEP", "PG", "CL", "KMB"],
29     "Pharma": ["PFE", "MRK", "JNJ", "BMY", "LLY"],
30     "Telecom": ["VZ", "T"],
31     "Airlines": ["DAL", "UAL", "AAL", "LUV"],
32     "Semis": ["INTC", "TXN", "QCOM", "MU"],
33     "Assurance": ["MET", "PRU", "AIG", "ALL", "TRV"],
34     "Homebuilders": ["DHI", "LEN", "PHM", "NVR"],
35     "Utilities": ["DUK", "SO", "NEE", "AEP", "D"],
36 }
37
38 import itertools
39 SECTOR_PAIRS = []
40 for sector, tickers in SECTOR_TICKERS.items():
41     for A, B in itertools.combinations(tickers, 2):
42         SECTOR_PAIRS.append((sector, A, B))
43 print(f"Univers candidat : {sum(len(v) for v in SECTOR_TICKERS.values())} tickers, "
44       f"{len(SECTOR_PAIRS)} paires intra-secteur", file=sys.stderr)
45
46 FORM_START, FORM_END = "2009-01-01", "2013-12-31" # etendu (vs 2014-2018 original)
47 TRAD_START, TRAD_END = "2014-01-01", "2024-12-31" # etendu (vs 2019-2024 original) -> 11 ans au lieu
48 de 6
49 COST = 0.0010
50 W = 60
51 N_MIX = 10
52 KAPPA, PI_STAR = 2.0, 0.5 # config de reference INCHANGE (pas de re-optimisation sur la fenetre de
53 test)
54
55 def mixture_density(x, muJ, sigJ, sigS, lam, dt=1.0):
56     k = np.arange(0, N_MIX + 1)
57     w = np.exp(-lam * dt) * (lam * dt) ** k / factorial(k)
58     mu_k = k * muJ
59     var_k = sigS ** 2 * dt + k * sigJ ** 2
60     dens = np.zeros_like(x, dtype=float)
61     for kk in range(N_MIX + 1):
62         dens += w[kk] * norm.pdf(x, loc=mu_k[kk], scale=np.sqrt(var_k[kk]))
63     return dens
64
65 def neg_loglik(p, resid):
66     lam, muJ, sigJ, sigS = p
67     if lam <= 0 or sigJ <= 0 or sigS <= 0:
68         return 1e10
69     d = np.clip(mixture_density(resid, muJ, sigJ, sigS, lam), 1e-12, None)
70     return -np.sum(np.log(d))
71
72
73 def fit_jump_mle(resid):
74     x0 = [0.05, np.mean(resid), np.std(resid) * 0.8, np.std(resid) * 0.5]
75     res = minimize(neg_loglik, x0, args=(resid,), method="Nelder-Mead",
76                  options=dict(maxiter=8000, xatol=1e-8, fatol=1e-10))

```

```

77     return res.x
78
79
80 def gauss_loglik(resid):
81     mu, sig = np.mean(resid), np.std(resid)
82     return np.sum(norm.logpdf(resid, mu, sig))
83
84
85 def posterior_jump_prob(x, muJ, sigJ, sigS, lam, dt=1.0):
86     k = np.arange(0, N_MIX + 1)
87     w = np.exp(-lam * dt) * (lam * dt) ** k / factorial(k)
88     mu_k = k * muJ
89     var_k = sigS ** 2 * dt + k * sigJ ** 2
90     num = np.zeros_like(x, dtype=float)
91     den = np.zeros_like(x, dtype=float)
92     for kk in range(N_MIX + 1):
93         c = w[kk] * norm.pdf(x, mu_k[kk], np.sqrt(var_k[kk]))
94         den += c
95         if kk >= 1:
96             num += c
97     return num / np.clip(den, 1e-12, None)
98
99
100 def get_pair_series(A, B, start, end):
101     yA = download_close(A, start, end)
102     yB = download_close(B, start, end)
103     df = pd.concat([yA, yB], axis=1, keys=["A", "B"]).dropna()
104     return df
105
106
107 def estimate_formation(df, fstart, fend):
108     logA, logB = np.log(df["A"]), np.log(df["B"])
109     fmask = (df.index >= fstart) & (df.index <= fend)
110
111     Xf = sm.add_constant(logB[fmask])
112     ols = sm.OLS(logA[fmask], Xf).fit()
113     alpha, beta = ols.params.iloc[0], ols.params.iloc[1]
114
115     spread = logA - alpha - beta * logB
116     spread_f = spread[fmask]
117     adf_p = adfuller(spread_f)[1]
118
119     s = spread_f.values
120     X2 = sm.add_constant(s[:-1])
121     ar1 = sm.OLS(s[1:], X2).fit()
122     c, phi = ar1.params[0], ar1.params[1]
123     resid = np.asarray(ar1.resid)
124     theta = -np.log(phi) if 0 < phi < 1 else np.nan
125     mu_s = c / (1 - phi)
126     sigma_s = resid.std() * np.sqrt(2 * theta / (1 - np.exp(-2 * theta))) if theta and theta > 0 else
        resid.std()
127
128     lam, muJ, sigJ, sigS_mle = fit_jump_mle(resid)
129     ll_mix = -neg_loglik([lam, muJ, sigJ, sigS_mle], resid)
130     ll_g = gauss_loglik(resid)
131     LR = 2 * (ll_mix - ll_g)
132     p_LR = 1 - chi2.cdf(LR, df=2)
133
134     return dict(alpha=alpha, beta=beta, theta=theta, mu_s=mu_s, sigma_s=sigma_s,
        lam=lam, muJ=muJ, sigJ=sigJ, adf_p=adf_p, LR=LR, p_LR=p_LR,
        spread=spread, phi=phi)
137
138
139 def simulate(spread_full, params, kappa, pi_star, use_filter, tstart, tend):
140     s_all = spread_full
141     phi = params["phi"]
142     mu_s = params["mu_s"]
143     theta, muJ, sigJ, sigS, lam = params["theta"], params["muJ"], params["sigJ"], params["sigma_s"],
        params["lam"]
144
145     mask = (s_all.index >= tstart) & (s_all.index <= tend)
146     idx = s_all.index[mask]
147     s = s_all.values
148     pos_full = np.where(mask)[0]
149     n = len(s)
150
151     z = pd.Series(s, index=s_all.index).rolling(W).apply(
152         lambda w: (w[-1] - w.mean()) / w.std() if w.std() > 0 else np.nan, raw=True
153     ).values
154
155     resid = np.full(n, np.nan)

```

```

156 resid[1:] = s[1:] - (mu_s + phi * (s[:-1] - mu_s))
157 pi_t = np.full(n, np.nan)
158 ok = ~np.isnan(resid)
159 pi_t[ok] = posterior_jump_prob(resid[ok], muJ, sigJ, sigS, lam)
160
161 position = 0
162 pnl = np.zeros(n)
163 n_trades = 0
164 durations = []
165 entry_i = None
166 wins = 0
167
168 for t in pos_full:
169     if t == 0 or np.isnan(z[t]):
170         continue
171     if position != 0:
172         pnl[t] = position * (s[t] - s[t - 1])
173         if (position == 1 and z[t] >= 0) or (position == -1 and z[t] <= 0):
174             pnl[t] -= 2 * COST
175             n_trades += 1
176             durations.append(t - entry_i)
177             trade_pnl = position * (s[t] - s[entry_i]) - 4 * COST
178             wins += int(trade_pnl > 0)
179             position = 0
180     if position == 0:
181         cand_long = z[t] <= -kappa
182         cand_short = z[t] >= kappa
183         if cand_long or cand_short:
184             allow = True
185             if use_filter and not np.isnan(pi_t[t]):
186                 allow = pi_t[t] < pi_star
187             if allow:
188                 position = 1 if cand_long else -1
189                 pnl[t] -= 2 * COST
190                 entry_i = t
191
192 ret = pd.Series(pnl[mask], index=idx)
193 hit_rate = wins / n_trades if n_trades > 0 else np.nan
194 return ret, dict(n_trades=n_trades, hit_rate=hit_rate,
195                 avg_dur=np.mean(durations) if durations else np.nan)
196
197
198 def sharpe(ret):
199     r = ret.dropna()
200     if r.std() == 0 or len(r) < 30:
201         return np.nan
202     return np.sqrt(252) * r.mean() / r.std()
203
204
205 def maxdd(ret):
206     cum = ret.cumsum()
207     return (cum - cum.cummax()).min()
208
209
210 def benjamini_hochberg(pvals, q=0.10):
211     pvals = np.asarray(pvals)
212     n = len(pvals)
213     order = np.argsort(pvals)
214     ranked = pvals[order]
215     thresh = (np.arange(1, n + 1) / n) * q
216     passed = ranked <= thresh
217     if not passed.any():
218         return np.zeros(n, dtype=bool)
219     kmax = np.max(np.where(passed))
220     cutoff = ranked[kmax]
221     return pvals <= cutoff
222
223
224 if __name__ == "__main__":
225     print(f"Fenetre de formation : {FORM_START} -> {FORM_END}")
226     print(f"Fenetre de trading : {TRAD_START} -> {TRAD_END}\n")
227
228     print(f"=== Screening ADF sur l'univers elargi ({len(SECTOR_PAIRS)} paires intra-secteur) ===")
229     FORM = {}
230     rows = []
231     skipped = []
232     for sector, A, B in SECTOR_PAIRS:
233         try:
234             df = get_pair_series(A, B, FORM_START, TRAD_END)
235             fmask_check = (df.index >= FORM_START) & (df.index <= FORM_END)
236             if fmask_check.sum() < 900: # pas assez d'historique sur la fenetre de formation

```

```

237         skipped.append((sector, A, B, f"historique insuffisant ({fmask_check.sum()} obs)"))
238         continue
239     p = estimate_formation(df, FORM_START, FORM_END)
240     FORM[(A, B)] = dict(p, df=df, sector=sector)
241     rows.append((sector, A, B, p["beta"], p["adf_p"], p["theta"], p["lam"], p["muJ"], p["p_LR"]
242                 ))
243     print(f"{sector:12s} {A}/{B:5s} beta={p['beta']:.7f} ADF_p={p['adf_p']:.4f} "
244           f"theta={p['theta']:.4f} lambda={p['lam']:.3f} LR_p={p['p_LR']:.4f}")
245     except Exception as e:
246         skipped.append((sector, A, B, str(e)))
247
248 if skipped:
249     print(f"\n{len(skipped)} paires ecartees (donnees insuffisantes) :")
250     for s in skipped:
251         print(f" {s[0]:12s} {s[1]}/{s[2]:5s} -> {s[3]}")
252
253 n_tested = len(rows)
254 pvals = [r[4] for r in rows]
255 selected_raw = [pvals[i] < 0.05 for i in range(len(pvals))]
256 selected_fdr = benjamini_hochberg(pvals, q=0.10)
257 print(f"\nADF p<0.05 (brut, sans correction) : {sum(selected_raw)}/{n_tested} paires")
258 print(f"ADF Benjamini-Hochberg (q=0.10) : {sum(selected_fdr)}/{n_tested} paires")
259 for i, r in enumerate(rows):
260     flag = "OK(FDR)" if selected_fdr[i] else ("ok(brut seulement)" if selected_raw[i] else "rejete")
261     print(f" {r[0]:10s} {r[1]}/{r[2]:5s} p={r[4]:.4f} -> {flag}")
262
263 # ===== Backtest portefeuille elargi (naive vs filtree) =====
264 selected_pairs = [(rows[i][1], rows[i][2]) for i in range(len(rows)) if selected_fdr[i]]
265 print(f"\n=== Backtest portefeuille elargi : {len(selected_pairs)} paires retenues (FDR q=0.10) ===")
266 print("Paires :", ", ".join(f"{A}/{B}" for A, B in selected_pairs))
267 print(f"kappa={KAPPA} pi*={PI_STAR} fenetre de trading {TRAD_START} -> {TRAD_END} (config INCHANGEE par rapport a 4.6, aucune re-optimisation)\n")
268
269 rets_naive, rets_filt, stats_naive, stats_filt = [], [], [], []
270 per_pair_rows = []
271 for A, B in selected_pairs:
272     p = FORM[(A, B)]
273     rn, sn = simulate(p["spread"], p, KAPPA, 1.0, False, TRAD_START, TRAD_END)
274     rf, sf = simulate(p["spread"], p, KAPPA, PI_STAR, True, TRAD_START, TRAD_END)
275     rets_naive.append(rn); rets_filt.append(rf)
276     stats_naive.append(sn); stats_filt.append(sf)
277     per_pair_rows.append((A, B, sn["n_trades"], sf["n_trades"], sharpe(rn), sharpe(rf)))
278     print(f" {A}/{B:5s} trades naive={sn['n_trades']:3d} trades filtree={sf['n_trades']:3d} "
279           f"Sharpe naive={sharpe(rn):6.3f} Sharpe filtree={sharpe(rf):6.3f}")
280
281 port_naive = pd.concat(rets_naive, axis=1).mean(axis=1)
282 port_filt = pd.concat(rets_filt, axis=1).mean(axis=1)
283
284 def agg_stats(stats_list):
285     return dict(n_trades=sum(s["n_trades"] for s in stats_list),
286               hit_rate=np.nanmean([s["hit_rate"] for s in stats_list]),
287               avg_dur=np.nanmean([s["avg_dur"] for s in stats_list]))
288
289 agg_n = agg_stats(stats_naive)
290 agg_f = agg_stats(stats_filt)
291
292 print("\n--- Resultats agregés (portefeuille equipondere) ---")
293 print(f"Naive : Sharpe={sharpe(port_naive):.3f} MaxDD={maxdd(port_naive):.3f} "
294       f"n_trades={agg_n['n_trades']} hit_rate={agg_n['hit_rate']*100:.1f}% duree_moy={agg_n['avg_dur']:.1f}j")
295 print(f"Filtree : Sharpe={sharpe(port_filt):.3f} MaxDD={maxdd(port_filt):.3f} "
296       f"n_trades={agg_f['n_trades']} hit_rate={agg_f['hit_rate']*100:.1f}% duree_moy={agg_f['avg_dur']:.1f}j")
297
298 def jk_mommel_test(rA, rB):
299     df = pd.concat([rA, rB], axis=1).dropna()
300     df.columns = ["A", "B"]
301     T = len(df)
302     muA, muB = df["A"].mean(), df["B"].mean()
303     sA, sB = df["A"].std(), df["B"].std()
304     sAB = df["A"].cov(df["B"])
305     theta_var = (1/T) * (2*sA**2*sB**2 - 2*sA*sB*sAB + 0.5*muA**2*sB**2 + 0.5*muB**2*sA**2 - (muA*muB/(sA*sB))*sAB**2)
306     z_stat = (sB*muA - sA*muB) / np.sqrt(theta_var)
307     p_val = 2 * (1 - norm.cdf(abs(z_stat)))
308     return z_stat, p_val
309
310 z_jk, p_jk = jk_mommel_test(port_filt, port_naive)
311 print(f"\nTest Jobson-Korkie/Mommel (filtree vs naive) : z={z_jk:.3f} p-value={p_jk:.4f}")

```

```

312 # sauvegarde pour reutilisation par le script de re-estimation glissante
313 import pickle
314 OUT_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "out")
315 os.makedirs(OUT_DIR, exist_ok=True)
316 with open(os.path.join(OUT_DIR, "selected_pairs.pkl"), "wb") as f:
317     pickle.dump(dict(selected_pairs=selected_pairs,
318                     FORM={k: {kk: vv for kk, vv in v.items() if kk != "df"} for k, v in FORM.
319                           items() if k in selected_pairs},
320                  port_naive=port_naive, port_filt=port_filt,
321                  agg_n=agg_n, agg_f=agg_f, per_pair_rows=per_pair_rows,
322                  sharpe_naive=sharpe(port_naive), sharpe_filt=sharpe(port_filt),
323                  maxdd_naive=maxdd(port_naive), maxdd_filt=maxdd(port_filt),
324                  z_jk=z_jk, p_jk=p_jk), f)
325 print(f"\nResultats sauvegardes dans {OUT_DIR}/selected_pairs.pkl")

```

A.7.3 — validated_config_train_test.py (calibration validation/test)

```

1  """
2  Calibration (kappa, pi*) sur un bloc de VALIDATION distinct du bloc de TEST final,
3  pour eviter le biais de selection multiple deja identifie par le DSR (section 4.7).
4  Univers : les 27 paires retenues par le screening FDR (pipeline.py).
5  - Formation (estimation des parametres OU+jump) : 2009-2013 (inchangee)
6  - Validation (calibration de kappa, pi*) : 2014-2018
7  - Test (evaluation, JAMAIS vue pendant la calibration) : 2019-2024
8  Regle de selection pre-enregistree : argmax du Sharpe du portefeuille FILTRE sur la
9  validation. Le resultat sur le test est rapporte tel quel, sans deuxieme passage.
10 """
11 import sys, os, pickle
12 import numpy as np
13 import pandas as pd
14 from scipy.stats import norm
15
16 sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
17 from expanded_universe_backtest import simulate, sharpe, maxdd
18
19 PKL_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), "out", "selected_pairs.pkl")
20 with open(PKL_PATH, "rb") as f:
21     saved = pickle.load(f)
22     FORM = saved["FORM"]
23     pairs = saved["selected_pairs"]
24
25 VALID_START, VALID_END = "2014-01-01", "2018-12-31"
26 TEST_START, TEST_END = "2019-01-01", "2024-12-31"
27
28 kappas = [1.5, 2.0, 2.5]
29 pistars = [0.3, 0.4, 0.5, 0.6, 0.7]
30
31 print(f"=== Calibration (kappa, pi*) sur validation {VALID_START}->{VALID_END}, "
32       f"test hors-echantillon {TEST_START}->{TEST_END} ({len(pairs)} paires) ===\n")
33
34 grid_valid = {}
35 for kappa in kappas:
36     for pistar in pistars:
37         rets = []
38         for A, B in pairs:
39             p = FORM[(A, B)]
40             r, _ = simulate(p["spread"], p, kappa, pistar, True, VALID_START, VALID_END)
41             rets.append(r)
42         port = pd.concat(rets, axis=1).mean(axis=1)
43         grid_valid[(kappa, pistar)] = sharpe(port)
44
45 print("Grille de validation (Sharpe filtree, 2014-2018) :")
46 for kappa in kappas:
47     line = " ".join(f"pi*={ps}:{grid_valid[(kappa,ps)]:.3f}" for ps in pistars)
48     print(f"  kappa={kappa}: {line}")
49
50 best_cfg = max(grid_valid, key=grid_valid.get)
51 print(f"\nConfig retenue (argmax Sharpe filtree sur VALIDATION uniquement) : "
52       f"kappa={best_cfg[0]}, pi*={best_cfg[1]} (Sharpe validation={grid_valid[best_cfg]:.3f})")
53
54 # --- Evaluation hors-echantillon (jamais vue pendant la calibration) ---
55 kappa_sel, pistar_sel = best_cfg
56 rets_naive, rets_filt, stats_naive, stats_filt = [], [], [], []
57 for A, B in pairs:
58     p = FORM[(A, B)]
59     rn, sn = simulate(p["spread"], p, kappa_sel, 1.0, False, TEST_START, TEST_END)
60     rf, sf = simulate(p["spread"], p, kappa_sel, pistar_sel, True, TEST_START, TEST_END)

```



```

61     rets_naive.append(rn); rets_filt.append(rf)
62     stats_naive.append(sn); stats_filt.append(sf)
63
64 port_naive = pd.concat(rets_naive, axis=1).mean(axis=1)
65 port_filt = pd.concat(rets_filt, axis=1).mean(axis=1)
66 n_naive = sum(s["n_trades"] for s in stats_naive)
67 n_filt = sum(s["n_trades"] for s in stats_filt)
68
69 print(f"\n== Resultat hors-echantillon sur le TEST ({TEST_START}->{TEST_END}), "
70       f"config figee kappa={kappa_sel} pi*={pistar_sel} ==")
71 print(f"Naive : Sharpe={sharpe(port_naive):.3f} MaxDD={maxdd(port_naive):.3f} n_trades={n_naive}")
72 print(f"Filtree : Sharpe={sharpe(port_filt):.3f} MaxDD={maxdd(port_filt):.3f} n_trades={n_filt}")
73
74
75 def jk_memmel_test(rA, rB):
76     df = pd.concat([rA, rB], axis=1).dropna()
77     df.columns = ["A", "B"]
78     T = len(df)
79     muA, muB = df["A"].mean(), df["B"].mean()
80     sA, sB = df["A"].std(), df["B"].std()
81     sAB = df["A"].cov(df["B"])
82     theta_var = (1/T) * (2*sA**2*sB**2 - 2*sA*sB*sAB + 0.5*muA**2*sB**2
83                        + 0.5*muB**2*sA**2 - (muA*muB/(sA*sB))*sAB**2)
84     z_stat = (sB*muA - sA*muB) / np.sqrt(theta_var)
85     p_val = 2 * (1 - norm.cdf(abs(z_stat)))
86     return z_stat, p_val
87
88 z_jk, p_jk = jk_memmel_test(port_filt, port_naive)
89 print(f"\nTest Jobson-Korkie/Memmel (filtree vs naive, sur TEST) : z={z_jk:.3f} p-value={p_jk:.4f}")

```

A.7.4 — exit_on_jump_filter.py (sortie anticipée sur saut détecté — résultat final)

```

1  """
2  Lever supplementaire, honnete celui-la aussi : au lieu de se contenter de bloquer l'ENTREE
3  quand pi_t >= pi*, la strategie filtree utilise aussi pi_t pour decider une SORTIE anticipée
4  en cours de position (des qu'un saut est detecte, on coupe, meme si le z-score n'est pas
5  revenu a zero). C'est precisement l'extension que le README du depot signale comme testee
6  dans backtest_4.6.py sans etre reconciliee avec les chiffres du rapport -- on la fait
7  proprement ici, calibre sur un bloc de VALIDATION distinct du bloc de TEST (meme protocole
8  que validated_config.py), sur l'univers elargi de 27 paires.
9  """
10 import sys, os, pickle
11 import numpy as np
12 import pandas as pd
13 from scipy.stats import norm
14
15 sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
16 from expanded_universe_backtest import posterior_jump_prob, sharpe, maxdd, COST, W
17
18 PKL_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), "out", "selected_pairs.pkl")
19 with open(PKL_PATH, "rb") as f:
20     saved = pickle.load(f)
21 FORM = saved["FORM"]
22 pairs = saved["selected_pairs"]
23
24 VALID_START, VALID_END = "2014-01-01", "2018-12-31"
25 TEST_START, TEST_END = "2019-01-01", "2024-12-31"
26
27
28 def simulate_exit_on_jump(spread_full, params, kappa, pi_star, pi_exit, use_filter, tstart, tend):
29     """Comme simulate() de pipeline.py, + sortie anticipée si pi_t >= pi_exit en cours de position
30     (uniquement actif si use_filter=True)."""
31     s_all = spread_full
32     phi = params["phi"]; mu_s = params["mu_s"]
33     theta, muJ, sigJ, sigS, lam = params["theta"], params["muJ"], params["sigJ"], params["sigma_s"],
34         params["lam"]
35
36     mask = (s_all.index >= tstart) & (s_all.index <= tend)
37     idx = s_all.index[mask]
38     s = s_all.values
39     pos_full = np.where(mask)[0]
40     n = len(s)
41
42     z = pd.Series(s, index=s_all.index).rolling(W).apply(
43         lambda w: (w[-1] - w.mean()) / w.std() if w.std() > 0 else np.nan, raw=True
44     ).values

```

```

44 resid = np.full(n, np.nan)
45 resid[1:] = s[1:] - (mu_s + phi * (s[:-1] - mu_s))
46 pi_t = np.full(n, np.nan)
47 ok = ~np.isnan(resid)
48 pi_t[ok] = posterior_jump_prob(resid[ok], muJ, sigJ, sigS, lam)
49
50 position = 0
51 pnl = np.zeros(n)
52 n_trades = 0
53 durations = []
54 entry_i = None
55 wins = 0
56 n_exit_jump = 0
57
58
59 for t in pos_full:
60     if t == 0 or np.isnan(z[t]):
61         continue
62     if position != 0:
63         pnl[t] = position * (s[t] - s[t - 1])
64         mean_revert_exit = (position == 1 and z[t] >= 0) or (position == -1 and z[t] <= 0)
65         jump_exit = use_filter and (not np.isnan(pi_t[t])) and (pi_t[t] >= pi_exit)
66         if mean_revert_exit or jump_exit:
67             pnl[t] -= 2 * COST
68             n_trades += 1
69             durations.append(t - entry_i)
70             trade_pnl = position * (s[t] - s[entry_i]) - 4 * COST
71             wins += int(trade_pnl > 0)
72             if jump_exit and not mean_revert_exit:
73                 n_exit_jump += 1
74             position = 0
75     if position == 0:
76         cand_long = z[t] <= -kappa
77         cand_short = z[t] >= kappa
78         if cand_long or cand_short:
79             allow = True
80             if use_filter and not np.isnan(pi_t[t]):
81                 allow = pi_t[t] < pi_star
82             if allow:
83                 position = 1 if cand_long else -1
84                 pnl[t] -= 2 * COST
85                 entry_i = t
86
87 ret = pd.Series(pnl[mask], index=idx)
88 hit_rate = wins / n_trades if n_trades > 0 else np.nan
89 return ret, dict(n_trades=n_trades, hit_rate=hit_rate, n_exit_jump=n_exit_jump,
90                 avg_dur=np.mean(durations) if durations else np.nan)
91
92
93 def port_sharpe(rets):
94     port = pd.concat(rets, axis=1).mean(axis=1)
95     return sharpe(port), port
96
97
98 print(f"=== Sortie anticipee sur detection de saut (pi_t >= pi_exit), univers elargi {len(pairs)}
99     paires ===")
100 print(f"Calibration sur VALIDATION {VALID_START}->{VALID_END}, test hors-echantillon {TEST_START}->{
101     TEST_END}\n")
102
103 kappas = [1.5, 2.0, 2.5]
104 pistars = [0.3, 0.4, 0.5]
105 piexits = [0.5, 0.6, 0.7, 0.8]
106
107 best = None
108 for kappa in kappas:
109     for pistar in pistars:
110         for piexit in piexits:
111             if piexit < pistar:
112                 continue
113             rets = []
114             for A, B in pairs:
115                 p = FORM[(A, B)]
116                 r, _ = simulate_exit_on_jump(p["spread"], p, kappa, pistar, piexit, True, VALID_START,
117                     VALID_END)
118                 rets.append(r)
119             sr, _ = port_sharpe(rets)
120             if best is None or sr > best[0]:
121                 best = (sr, kappa, pistar, piexit)
122
123 print(f"Meilleure config sur VALIDATION (argmax Sharpe filtree) : "
124       f"kappa={best[1]}, pi*={best[2]}, pi_exit={best[3]} (Sharpe validation={best[0]:.3f})")

```

```

122 _, kappa_sel, pistar_sel, piexit_sel = best
123
124
125 rets_naive, rets_filt, stats_naive, stats_filt = [], [], [], []
126 for A, B in pairs:
127     p = FORM[(A, B)]
128     rn, sn = simulate_exit_on_jump(p["spread"], p, kappa_sel, 1.0, 1.0, False, TEST_START, TEST_END)
129     rf, sf = simulate_exit_on_jump(p["spread"], p, kappa_sel, pistar_sel, piexit_sel, True, TEST_START,
        TEST_END)
130     rets_naive.append(rn); rets_filt.append(rf)
131     stats_naive.append(sn); stats_filt.append(sf)
132
133 port_naive = pd.concat(rets_naive, axis=1).mean(axis=1)
134 port_filt = pd.concat(rets_filt, axis=1).mean(axis=1)
135 n_naive = sum(s["n_trades"] for s in stats_naive)
136 n_filt = sum(s["n_trades"] for s in stats_filt)
137 n_exit_jump_total = sum(s["n_exit_jump"] for s in stats_filt)
138
139 print(f"\n=== Resultat hors-echantillon TEST ({TEST_START}->{TEST_END}), "
140       f"kappa={kappa_sel} pi*={pistar_sel} pi_exit={piexit_sel} ===")
141 print(f"Naive : Sharpe={sharpe(port_naive):.3f} MaxDD={maxdd(port_naive):.3f} n_trades={n_naive}")
142 print(f"Filtree : Sharpe={sharpe(port_filt):.3f} MaxDD={maxdd(port_filt):.3f} n_trades={n_filt} "
143       f"(dont {n_exit_jump_total} sorties anticipées sur detection de saut)")
144
145
146 def jk_mommel_test(rA, rB):
147     df = pd.concat([rA, rB], axis=1).dropna()
148     df.columns = ["A", "B"]
149     T = len(df)
150     muA, muB = df["A"].mean(), df["B"].mean()
151     sA, sB = df["A"].std(), df["B"].std()
152     sAB = df["A"].cov(df["B"])
153     theta_var = (1/T) * (2*sA**2*sB**2 - 2*sA*sB*sAB + 0.5*muA**2*sB**2
154                        + 0.5*muB**2*sA**2 - (muA*muB/(sA*sB))*sAB**2)
155     z_stat = (sB*muA - sA*muB) / np.sqrt(theta_var)
156     p_val = 2 * (1 - norm.cdf(abs(z_stat)))
157     return z_stat, p_val
158
159 z_jk, p_jk = jk_mommel_test(port_filt, port_naive)
160 print(f"\nTest Jobson-Korkie/Mommel (filtree vs naive, sur TEST) : z={z_jk:.3f} p-value={p_jk:.4f}")

```

A.8 — Reproduire ces résultats

```

1 cd code
2 pip install requests pandas numpy scipy statsmodels
3 python3 expanded_universe_backtest.py # criblage FDR + backtest agrege
4 python3 validated_config_train_test.py # calibration validation/test
5 python3 exit_on_jump_filter.py # resultat final : Sharpe filtree = 0.343, 442 trades

```

yahoo_dl.py télécharge les prix via l'API chart de Yahoo Finance directement (sans yfinance, qui échouait pour des raisons réseau dans l'environnement où cette annexe a été produite — yfinance standard devrait fonctionner directement sur une machine sans cette contrainte réseau).